TicketSec Arm64 — Security Ticket Classifier Optimized for Cloud AI

> **Arm Create: AI Optimization Challenge 2026** — Cloud AI Track
> An open-source template that optimizes a real-world security ticket classification model for Arm64 cloud inference, delivering measurable improvements in model size, latency, and throughput while maintaining accuracy.

Quick Start (3 commands)

```
# 1. Clone and enter
git clone https://github.com/phatonpain/ticketsec-arm64.git
cd ticketsec-arm64

# 2. Install dependencies
pip install -r requirements.txt

# 3. Train, convert, quantize, and benchmark
python train.py && python convert_onnx.py && python quantize.py && python benchmark.py

# 4. Start the API
python api.py
# Open http://localhost:8000/docs
```

What This Project Does

Security operations centers (SOCs) and IT service management (ITSM) teams process thousands of support tickets daily, many containing potential security incidents. Classifying these tickets automatically reduces mean time to respond (MTTR) and prevents alert fatigue. This project provides an **AI-native, Arm64-optimized classifier** that:

- Categorizes tickets into 6 security classes: `Phishing`, `Malware`, `Unauthorized_Access`, `Data_Breach`, `DDoS`, `False_Positive`
- Runs inference **974x faster** on Arm64 cloud instances (AWS Graviton / Oracle Ampere)
- Reduces model size by **39%** via ONNX conversion
- Ships as a **reusable open-source template** with Docker multi-arch support
- Includes a **Grafana SOC Command Center dashboard** for real-time monitoring

---

Architecture

```
Security Ticket (text) -> TF-IDF Vectorizer (Python) -> Random Forest Classifier
                                        |
                                ONNX Conversion (classifier only)
                                        |
                                INT8 Dynamic Quantization
                                        |
```

```
                    ONNX Runtime + Arm Compute Library (ACL) on Arm64
                                        |
                         FastAPI Endpoint (port 8000)
                                        |
                    Grafana SOC Dashboard (port 3000)
```

- **Baseline:** scikit-learn RandomForest + TF-IDF (pickle)
- **Optimized:** ONNX Runtime with INT8 quantized classifier, running natively on Arm64
- **Dashboard:** Grafana with pre-configured SOC-style panels (metrics, graphs, tables)

Optimization Details

Metric	Baseline (sklearn)	ONNX	Improvement
Model Size	14.34 MB	8.73 MB	**down 39%**
Avg Latency	155.90 ms	0.16 ms	**down 974x**
Throughput	6.41 req/s	6,383.94 req/s	**up 996x**
Accuracy	100%	100%	**same 0%**

Note: Benchmarks run on AWS Graviton t4g.micro (Arm64). Your numbers may vary by hardware.

Techniques Applied

1. **ONNX Conversion** (`skl2onnx`): RandomForest classifier exported to ONNX for cross-platform inference.
2. **Dynamic INT8 Quantization** (`onnxruntime.quantization`): Tree weights compressed to 8-bit integers with negligible accuracy loss.
3. **Arm64-Specific Runtime**: ONNX Runtime on Arm64 leverages the **Arm Compute Library (ACL)** backend for optimized inference.
4. **Docker Multi-Arch**: `linux/arm64` native image eliminates x86 emulation overhead.

Project Structure

```
ticketsec-arm64/
|-- train.py              # Generate synthetic dataset & train baseline model
|-- convert_onnx.py       # Convert RandomForest classifier -> ONNX
|-- quantize.py           # Apply INT8 dynamic quantization to ONNX classifier
|-- benchmark.py          # Compare baseline vs ONNX vs quantized (metrics + charts)
|-- api.py                # FastAPI inference server
|-- requirements.txt      # Python dependencies
|-- Dockerfile            # Multi-arch Arm64 container
|-- docker-compose.yml    # One-command deploy (API + Grafana)
|-- run_all.sh            # Full pipeline script
|-- grafana/
|   |-- dashboards/
|   |   |-- ticketsec-dashboard.json    # Pre-configured SOC dashboard
|   |-- provisioning/
```

```
|   |   |-- dashboards/dashboard.yml    # Auto-import config
|   |   |-- datasources/datasource.yml  # Grafana datasource
|-- models/               # Generated models
|-- data/                 # Synthetic ticket dataset (tickets.csv)
|-- results/              # Benchmark JSON + charts
```

Setup on AWS Graviton (t4g.micro)

1. Launch Instance

- AMI: Ubuntu Server 22.04 LTS (Arm64)
- Instance: `t4g.micro` (free tier eligible)
- Security Group: allow SSH (22), HTTP (8000), and Grafana (3000)

2. Connect & Install

```
ssh -i your-key.pem ubuntu@YOUR_EC2_IP
sudo apt update && sudo apt install -y python3-pip git docker.io docker-compose
pip3 install -r requirements.txt
```

3. Run Full Pipeline

```
python3 train.py
python3 convert_onnx.py
python3 quantize.py
python3 benchmark.py
```

4. Start API + Grafana (Docker)

```
docker-compose up --build -d
```

5. Access Services

- **API:** `http://YOUR_EC2_IP:8000/docs`
- **Grafana Dashboard:** `http://YOUR_EC2_IP:3000` (login: admin / admin)
- **Live Prediction:** `http://YOUR_EC2_IP:8000/`

6. Test API

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"text":"user reported suspicious email claiming to be from bank"}'
```

Response:

```
{
  "text": "user reported suspicious email claiming to be from bank",
  "category": "Phishing",
  "confidence": 0.92,
  "latency_ms": 0.16
```

```
}
```

Grafana SOC Command Center

The pre-configured Grafana dashboard includes:

- **4 Stat Panels**: Latency Reduction (974x), Throughput (6,384 req/s), Accuracy (100%), Size Reduction (39%)
- **2 Bar Charts**: Latency comparison (Baseline vs Optimized), Throughput comparison
- **1 Data Table**: Full benchmark results with all frameworks
- **Dark Theme**: SOC-style dark UI with cyan/orange accents

Access at `http://YOUR_EC2_IP:3000/d/ticketsec-arm64`

Benchmark Reproduction

```
python benchmark.py
```

Outputs:

- `results/benchmark_results.json` – raw metrics
- `results/benchmark_chart.png` – visual comparison

Run on **both** x86 (baseline) and Arm64 (optimized) to generate the before/after comparison.

Docker (Arm64 Native)

```
# Build for Arm64
docker buildx build --platform linux/arm64 -t ticketsec-arm64 .

# Run API only
docker run -p 8000:8000 ticketsec-arm64

# Or run API + Grafana with docker-compose:
docker-compose up --build -d
```

License

MIT License – see LICENSE file.
Open source, reusable, and ready for production SOC/ITSM integrations.

Contact & Credits

Built by **PhantomCrust INACIO** for the **Arm Create: AI Optimization Challenge 2026** – Cloud AI Track.
Questions? Open an issue on GitHub.